

Reading App.tsx (3/3)

Persistence and Review

Adding Persistence (Extra)

Saving data is an extra feature of this example (the JavaScript and TypeScript versions do not save).

Without it, a page reload loses all the grades.

We add it in three steps:

1. Save the grades to `localStorage`.
2. Load the saved grades, and **check** them.
3. Connect both to the state of `GradeCalculatorApp`.

1. Saving

```
function saveGrades(grades: Grade[]): void {  
  try {  
    localStorage.setItem(STORAGE_KEY, JSON.stringify(grades));  
  } catch {  
    // storage is full or blocked: the app still works  
  }  
}
```

- `JSON.stringify` converts the array to a string, because `localStorage` stores only strings.
- `catch { ... }` without a variable is allowed when we do not need the error object.

| 2. Loading Saved Grades

When the app starts, it reads the saved string and converts it back into JavaScript data with

`JSON.parse`.

- Saved data might be missing fields or have values of the wrong type, so the app checks each item before using it as a `Grade`.
- TypeScript types alone cannot check data at runtime.

Two small pieces of syntax appear in the next slides:

- `unknown` means "a value of any type that we have not checked yet". Unlike `any`, TypeScript does not let us use an `unknown` value until we check it.
- `a ?? b` means "use `a`, but if `a` is `null` or `undefined`, use `b`". If nothing is saved, `getItem` returns `null`, so we use `[]` (an empty array in JSON).

Check whether a value is a Grade

`isGrade` checks that a value has the fields a grade needs: a numeric `id`, a text `name`, and numeric `score` and `weight`.

- The `value is Grade` return type is a **type guard**: when the function returns `true`, TypeScript knows the value is a `Grade`.

```
function isGrade(value: unknown): value is Grade {  
  if (typeof value !== 'object' || value === null) return false;  
  const g = value as Record<string, unknown>;  
  return (  
    typeof g.id === 'number' &&  
    typeof g.name === 'string' &&  
    typeof g.score === 'number' &&  
    typeof g.weight === 'number'  
  );  
}
```

Load the saved data and keep valid grades

`loadGrades` parses the saved string, then uses `isGrade` to keep only valid grade items.

- If nothing has been saved, the data is invalid JSON, or the top-level value is not an array, it returns an empty list.

```
function loadGrades(): Grade[] {  
  try {  
    const parsed: unknown =  
      JSON.parse(localStorage.getItem(STORAGE_KEY) ?? '[]');  
    return Array.isArray(parsed) ? parsed.filter(isGrade) : [];  
  } catch {  
    return []; // broken JSON, or storage is not available  
  }  
}
```

3. Connect to the State

Only two lines of `GradeCalculatorApp` change:

```
function GradeCalculatorApp() {  
  // The function runs only once, when the state is first created  
  const [grades, setGrades] = useState<Grade[]>(loadGrades);  
  
  // Save to localStorage whenever grades change  
  useEffect(() => {  
    saveGrades(grades);  
  }, [grades]);  
}
```

- `useState(loadGrades)` receives the function itself; React calls it once for the first value.
- The effect runs after a render in which `grades` changed.

Try It: Break the Code on Purpose

Change one thing at a time, rebuild, and watch what happens.

1. In `addGrade`, replace the `setGrades(...)` call with `grades.push(newGrade)`.
 - There is no type error, but **the screen does not update**: React is never told that the state changed.
2. Remove `key={grade.id}` in `GradeList`.
 - The page still works, but the browser console shows a warning about a missing `key`.

3. Change the dependency array `[grades]` to `[]` in `useEffect`.

- The grades are saved only once at the start, so a reload loses your new grades.

4. Misspell `grade.score` as `grade.scor` in `GradeItem`.

- `npm run check` reports a type error.
- But `esbuild` alone **builds successfully**: it does not type-check.

Compare the two scripts after the typo in step 4:

```
bash run-npm-nocheck.sh  # builds app.js: no type check, the bug is still there
bash run.sh              # stops with a type error: nothing is built
```

This is why `npm run build` runs `check` first. Fix the typo before you continue.

Judge It with the Same Questions

Remember the three criteria from the JavaScript example:

1. Can the calculation be tested without a browser? **Yes.** `calculateFinalGrade` is a pure function, and it does not need React.
2. Can the UI change without touching the logic? **Yes.** The components only describe the UI. The logic is in the pure functions and in the parent.
3. Is there a single source of truth for the state? **Yes.** `grades` lives in `GradeCalculatorApp`. The list, the result, and the saved data are all derived from it.

Compare the Versions

	JavaScript / TypeScript	React
Find the UI elements	<code>getElementById</code> (TypeScript also checks the element type)	not needed: JSX describes the UI
Update the screen	call <code>render()</code> after every change	automatic after the state changes
Where is the state?	a variable or a class field	<code>useState</code>

UI = f(state): we change the state, and React updates the screen.

What We Still Pay For

- A build step (npm, esbuild) and about 1 MB of library code.
- New concepts to learn: components, props, state, effects, keys.
- TypeScript types still disappear at runtime, so outside data (localStorage, network) must be checked.